

Eventual Timed Automata

Deepak D'Souza and Raj Mohan M

Department of Computer Science and Automation,
Indian Institute of Science, Bangalore 560012, India.
Email: {deepakd,raj}@csa.iisc.ernet.in

Abstract. We study the class of timed automata called *eventual timed automata* (ETA's) obtained using guards based on the operator $\Diamond$. In this paper we show that ETA's form a decidable class of timed automata via a flattening to non-recursive ETA's followed by a reduction to 1-clock alternating timed automata. We also study the expressiveness of the class of ETA's and show that they compare favourably with other classes in the literature. Finally we show that class obtained using the dual operator $\Diamond$ is also decidable, though the two operators together lead to an undecidable class of languages.

1 Introduction

The timed automata of Alur and Dill [AD94] are a popular model of time dependent systems. While they are very expressive and have a decidable emptiness problem, they suffer from the fact that they are not closed under complement and have an undecidable language inclusion problem. As a result of non-closure under complementation, they also do not admit natural monadic second order logic characterizations. In an attempt to describe a robust class of timed automata which are closed under boolean operations and which have more robust logical properties, the generic class of Input Determined Automata (IDA's) were identified in [DT04]. In contrast to a timed automaton, an IDA does not have explicit clocks but uses distance operators whose meaning is determined solely by the given timed word and a position in it. These generic classes of automata have several desirable properties. They are determinizable, closed under boolean operations, admit natural monadic second order logic characterizations, and have expressively complete timed temporal logics.

In this paper we focus on the class of IDA's based on the input determined operator $\Diamond$, and call it *eventual timed automata* or ETA's. The operator $\Diamond_a$, where a is a letter of the alphabet, identifies a set of distances to future positions in the timed word which are labeled a . An eventual timed automaton can use guards of the form $(\Diamond_a \in I)$ which is then true at a point in a timed word provided there is a position in the future where an a occurs and the distance to that point lies in the interval I . In general, the class of *recursive* ETA's can make use of operators of the form $\Diamond_{\mathcal{B}}$, where $\mathcal{B}$ is in turn such an ETA which accepts timed words *at* a designated position in them.

The class of recursive ETA's are an interesting class of timed automata for reasons other than that they inherit the useful properties of generic IDA's listed above. The well known timed temporal logic Metric Temporal Logic (MTL) [Koy90,AFH96,OW05] corresponds to the first order fragment of recursive ETA's. That is, MTL is expressively equivalent to the first order fragment of the timed MSO which characterizes recursive ETA's [DT04]. Further, they form an orthogonal class of timed languages with respect to Alur-Dill timed automata, since they can express some languages not expressible by timed automata [DT04]. However nothing much was known about decision procedures for this class of languages.

In this paper we address this issue and show that recursive ETA's form a decidable class of timed languages. We do this in two steps. We first show that recursive ETA's can be "flattened" to a non-recursive one over an extended alphabet. We then show that the emptiness problem for ETA's can be decided by showing how they can be simulated by 1-clock alternating timed automata which were recently shown to be decidable [LW05,OW05].

We also study the expressiveness of this class more closely, and show that it is indeed incomparable with the class of timed automata in that there is also a language which timed automata can express but a recursive ETA cannot. The class of recursive ETA's can also be seen to be strictly more expressive than the class of recursive Event Predicting Automata [AFH94,HRS98] which are another class with robust logical properties [DT04]. This, together with the properties described above, make the class of recursive ETA's one of the most expressive classes of timed automata of its kind in the literature.

Finally we show that the class of automata obtained by considering the dual operator $\diamond$ is also decidable. We do this by reducing its emptiness problem to that of ETA's. However, the two operators taken together lead to an undecidable class of automata.

Some proofs are omitted due to lack of space and can be found in the full version of this paper [DM05].

2 Eventual Timed Automata

For an alphabet A we use A^* to denote the set of finite words over A , and $|w|$ to denote the length of a word w . The set of non-negative and positive real numbers will be denoted by $\mathbb{R}_{\geq 0}$ and $\mathbb{R}_{> 0}$ respectively.

A (finite) *timed word* over an alphabet Σ is an element of $(\Sigma \times \mathbb{R}_{> 0})^*$ of the form $\sigma = (a_0, t_0)(a_1, t_1) \cdots (a_n, t_n)$, satisfying $t_i < t_{i+1}$ for all $i \in \{0, \dots, n-1\}$. Wherever convenient, we will also use the representation of the timed word σ above as (w, τ) where $w = a_0 \cdots a_n$ and $\tau = t_0 \cdots t_n$. We denote the set of all timed words over Σ by $T\Sigma^*$.

We will use rational bounded intervals to specify timing constraints. These intervals can be open or closed, and we allow ∞ as an open right end. These intervals denote a subset of reals in the usual manner – for example $[2, \infty)$ denotes the set $\{t \in \mathbb{R}_{\geq 0} \mid 2 \leq t\}$. The set of all intervals is denoted $\mathcal{I}_{\mathbb{Q}}$.

An eventual timed automaton is essentially a timed automaton which has no explicit clocks, but uses atomic guards of the form “ $\diamond_a \in I$ ”, which assert that with respect to the current position in a timed word there is an occurrence of an action a at some point in the future, such that the distance to that point lies in the interval I . To define these automata more formally, we begin by defining the guards used by them. For an alphabet Σ , we use $\mathcal{G}(\Sigma)$ to denote the set of guards given by the syntax $g ::= \top \mid \diamond_a \in I \mid \neg g \mid g \vee g \mid g \wedge g$, where $a \in \Sigma$ and $I \in \mathcal{I}_{\mathbb{Q}}$. The guards are interpreted at the “action points” in a timed word σ (we call these *positions* and denote them by natural numbers between 0 and $|\sigma| - 1$). Let $\sigma = (a_0, t_0) \cdots (a_n, t_n)$ and $i \in \{0, \dots, |\sigma| - 1\}$. Then σ at position i satisfies the guard $\diamond_a \in I$, written $\sigma, i \models \diamond_a \in I$, iff there exists $j \geq i$ such that $a_j = a$ and $t_j - t_i \in I$. The guard $\top$ is always true, and the boolean operators are interpreted in the expected manner.

An *eventual timed automaton (ETA)* over Σ is of the form $\mathcal{A} = (Q, s, \delta, F)$ where Q is a finite set of states, $s \in Q$ is the start state, δ is the transition relation and is a finite subset of $Q \times \Sigma \times \mathcal{G}(\Sigma) \times Q$, and $F \subseteq Q$ is a set of final states. A run of $\mathcal{A}$ on a timed word $\sigma = (a_0, t_0) \cdots (a_n, t_n)$ is a sequence of states $q_0, \dots, q_{n+1}$ satisfying

- $q_0 = s$,
- for each $i \in \{0, \dots, n\}$ there exist guards g_i such that $(q_i, a_i, g_i, q_{i+1}) \in \delta$ and $\sigma, i \models g_i$.

The run is accepting if $q_{n+1} \in F$. The language of timed words accepted by $\mathcal{A}$ is denoted $L(\mathcal{A})$ and defined to be the set of all timed words in $T\Sigma^*$ on which $\mathcal{A}$ has an accepting run.

The figure below shows an example of a simple ETA with a single state that is both the start and the final state. It accepts the language L_0 of all timed words of a ’s in which there are no two a ’s at a distance of 1 time unit apart.

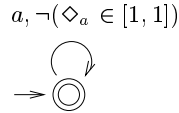


Fig. 1. An example ETA

It will be convenient to recall the general class of input determined automata (IDA) defined in [DT04]. An IDA over an alphabet Σ has a structure similar to an ETA except that it is parameterized by a set of “input determined” operators Op . Each operator $\Delta \in Op$ has a semantic function $\llbracket \Delta \rrbracket$ which we assume in this paper associates a set of distances with a given timed word σ and a position i in it. Thus $\llbracket \Delta \rrbracket(\sigma, i) \subseteq \mathbb{R}_{\geq 0}$. The atomic guards in an IDA are now of the form $(\Delta \in I)$ and the truth of the guard at position i in a timed word σ is given by $\sigma, i \models \Delta \in I$ iff $\llbracket \Delta \rrbracket(\sigma, i) \cap I \neq \emptyset$. The timed language accepted by an IDA $\mathcal{A}$

over an alphabet Σ and a set of operators Op is denoted $L(\mathcal{A})$ and is defined as for ETA's, with the guards interpreted as described above.

Thus the class of ETA's over an alphabet Σ defined above is nothing but the class of IDA's corresponding to the set of operators $\{\diamond_a \mid a \in \Sigma\}$ (we denote this class by $IDA(\diamond)$) with the semantic function $\llbracket \diamond_a \rrbracket$ given by

$$\llbracket \diamond_a \rrbracket(\sigma, i) = \{\tau(j) - \tau(i) \mid \sigma = (w, \tau), j \geq i, \text{ and } w(j) = a\}.$$

We now introduce the *recursive* version of ETA's. The main idea is to index the $\diamond$ operator with an *automaton* $\mathcal{B}$ and use the operator $\diamond_{\mathcal{B}}$ to mean the set of distances to the points in the future where the automaton $\mathcal{B}$ “accepts”. To make this notion more precise, we first introduce the notion of a *floating* language. A *floating language* over Σ is a set of *floating timed words* over Σ which in turn are pairs of the form (σ, i) where σ is a timed word over Σ and i is a position in σ . We will represent a “floating” word (σ, i) as a timed word over the extended alphabet $\Sigma \times \{0, 1\}$. Thus a timed word ν over $\Sigma \times \{0, 1\}$ represents the floating word (σ, i) , iff $\nu = (w, v, \tau)$, with $v \in \{0, 1\}^*$ with a *single* 1 in the i -th position, and $\sigma = (w, \tau)$. We use fw to denote the (partial) map which given a timed word ν over $\Sigma \times \{0, 1\}$ returns the floating word (σ, i) corresponding to ν , and extend it to apply to timed languages over $\Sigma \times \{0, 1\}$ in the natural way.

We can now define recursive ETA's and the timed languages they accept as follows. The class of *recursive eventual timed automata* (recursive ETA's) over the alphabet Σ is the union of the classes of level i recursive eventual timed automata over Σ (for $i \geq 0$) which are defined inductively below:

- A *level 0 recursive ETA* over Σ is an ETA $\mathcal{A}$ over Σ that uses only the guard $\top$. It accepts the language $L(\mathcal{A})$ as defined for ETA's.
- Similarly, a *level 0 recursive floating ETA* over Σ is an ETA $\mathcal{B}$ over the alphabet $\Sigma \times \{0, 1\}$ that uses only the guard $\top$. It accepts the floating language $L^f(\mathcal{B})$ defined to be $fw(L(\mathcal{B}))$.
- Let D be a finite collection of recursive floating ETA of level i or less over Σ , and consider the set of operators $Op = \{\diamond_{\mathcal{B}} \mid \mathcal{B} \in D\}$. We define the semantics of $\diamond_{\mathcal{B}}$ in Op by:

$$\llbracket \diamond_{\mathcal{B}} \rrbracket(\sigma, i) = \{\tau(j) - \tau(i) \mid \sigma = (w, \tau), j \geq i, \text{ and } (\sigma, j) \in L^f(\mathcal{B})\}.$$

A *level $i + 1$ recursive ETA* over Σ is then an IDA $\mathcal{A}$ over (Σ, Op) , and accepts the language $L(\mathcal{A})$ as defined for IDA's. We also require that $\mathcal{A}$ should use at least one operator of the form $\diamond_{\mathcal{B}}$ for a recursive floating ETA $\mathcal{B}$ of level i .

- To define level $i + 1$ recursive floating ETA's, let D be a finite collection of recursive floating ETA's of level i or less over Σ , and consider the set of operators $Op = \{\diamond_{\mathcal{B}} \mid \mathcal{B} \in D\}$. For floating automata the operator $\diamond$ works slightly differently in that it ignores the extended part of the alphabet. We define the semantics of $\diamond_{\mathcal{C}}$ when used in a floating automaton as follows. Let σ' be a timed word over $\Sigma \times \{0, 1\}$. Then

$$\llbracket \diamond_{\mathcal{C}} \rrbracket(\sigma', i) = \{\tau(j) - \tau(i) \mid \sigma' = (w, v, \tau), j \geq i, \text{ and } ((w, \tau), j) \in L^f(\mathcal{C})\}.$$

A *level $i + 1$ recursive floating ETA* over Σ is then an IDA $\mathcal{B}$ over $(\Sigma \times \{0, 1\}, Op)$, and accepts the floating language $L^f(\mathcal{B})$ defined to be $fw(L(\mathcal{B}))$. Once again we require that $\mathcal{B}$ should use at least one operator of the form $\diamond_c$ for a recursive floating ETA $\mathcal{C}$ of level i .

Below we give an example of a recursive ETA over the alphabet $\{a, b, c\}$.

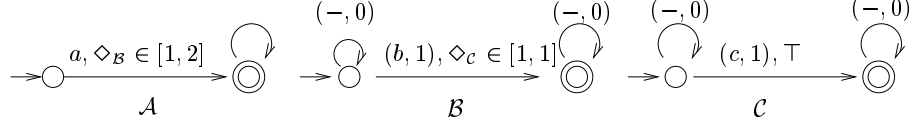


Fig. 2. A level 2 recursive ETA.

Fig. 2 shows a level 2 recursive ETA $\mathcal{A}$ which uses a level 1 floating automaton $\mathcal{B}$, which in turn calls a level 0 floating automaton $\mathcal{C}$. $\mathcal{A}$ accepts timed words σ which start with an a at some time t , followed eventually by a b at time t' such that $t' - t \in [1, 2]$, and a c at time $t'' = t' + 1$. For example, the timed word $(a, 0.2)(a, 1.2)(b, 1.4)(c, 2.4)$ satisfies all the conditions given above and therefore belongs to $L(\mathcal{A})$.

Before we close this section, we recall some of the results of [DT04] specialized to the case of recursive ETA's. We recall briefly the timed monadic second order logic $\text{rec-TMSO}(\Sigma)$ based on the operator $\diamond$, which characterizes the class of timed languages defined by recursive ETA's. The syntax of the logic is given by

$$\varphi ::= Q_a(x) \mid \diamond_\psi(x) \in I \mid x \in X \mid x < y \mid \neg\varphi \mid (\varphi \vee \varphi) \mid \exists x\varphi \mid \exists X\varphi.$$

In the predicate $\diamond_\psi(x) \in I$, we require ψ to be a $\text{rec-TMSO}(\Sigma)$ formula with a single free variable z , x a first-order variable, and $I \in \mathcal{I}_{\mathbb{Q}}$.

The logic is interpreted over timed words in $T\Sigma^*$. For formulas with free variables we use an interpretation $\mathbb{I}$ which maps variables to positions in a timed word. The semantics of the predicate “ $\diamond_\psi(x) \in I$ ” is defined inductively as follows. If ψ is a formula which uses no $\diamond$ predicates, then the satisfaction relation $\sigma, \mathbb{I} \models \psi$ is defined as for standard MSO. Inductively, assuming the semantics of ψ has already been defined, $\diamond_\psi$ is interpreted as an input determined operator as follows:

$$\llbracket \diamond_\psi \rrbracket(\sigma, i) = \{\tau(j) - \tau(i) \mid \sigma = (w, \tau), j \geq i, \text{ and } \sigma, [j/z] \models \psi\}.$$

Then we say:

$$\sigma, \mathbb{I} \models \diamond_\psi(x) \in I \quad \text{iff} \quad \llbracket \diamond_\psi \rrbracket(\sigma, \mathbb{I}(x)) \cap I \neq \emptyset.$$

A sentence φ in $\text{rec-TMSO}(\Sigma)$ defines the language $L(\varphi) = \{\sigma \in T\Sigma^\omega \mid \sigma \models \varphi\}$.

Theorem 1 ([DT04]). *The class of recursive ETA's over an alphabet Σ satisfies the following properties:*

1. *They are determinizable and closed under boolean operations.*
2. *The timed monadic second order logic $\text{rec-TMSO}(\Sigma)$ is expressively equivalent to them.*
3. *The timed temporal logic MTL is expressively equivalent to the first order fragment of $\text{rec-TMSO}(\Sigma)$.* $\square$

3 Deciding Emptiness for ETA's

In this section we give a decision procedure for the language emptiness problem for ETA's. We do this by showing how to translate an ETA to a 1-clock alternating timed automaton, the emptiness problem for which has been shown to be decidable [LW05,OW05].

We first recall the definition of a 1-clock alternating timed automaton. Let $\mathcal{G}_x$ denote the set of guards over the clock x given by the following syntax: $g ::= \top \mid x \in I \mid \neg g \mid g \vee g$. A valuation for the clock x , $d \in \mathbb{R}_{\geq 0}$, satisfies $x \in I$, written $d \models x \in I$, iff $d \in I$. The other guards are interpreted in the expected manner. For $X \subseteq \{x\}$, we use $d[0/X]$ to denote the valuation d or 0, depending on whether $X = \emptyset$ or $\{x\}$.

A 1-clock *alternating timed automaton* (ATA) over an alphabet Σ is of the form $\mathcal{T} = (Q, s, \Delta, F)$ where Q is a finite set of states, $s \in Q$ is the initial state, $\Delta \subseteq Q \times \Sigma \times \mathcal{G}_x \times (2^{\{\emptyset, \{x\}\} \times Q} - \{\emptyset\})$ is the transition relation, and $F \subseteq Q$ is the set of final states.

An *extended state* of $\mathcal{T}$ is a pair of the form (q, d) where $q \in Q$ and $d \in \mathbb{R}_{\geq 0}$ is a valuation for x . A *configuration* of $\mathcal{T}$ is a finite collection of extended states. To describe a run of the ATA over a timed word it is convenient to first define a consecution relation between configurations. Let $C = \{(q_0, d_0), \dots, (q_k, d_k)\}$ be a configuration of $\mathcal{T}$. Let $t \in \mathbb{R}_{\geq 0}$ and $a \in \Sigma$. Then a configuration D is an (a, t) -successor of C iff D is the union of configurations $D_0, \dots, D_k$ which satisfy: there exist guards g_i and sets of reset-state pairs $T_i = \{(r_0^i, p_0^i), \dots, (r_{n_i}^i, p_{n_i}^i)\}$ such that

- $(q_i, a, g_i, T_i) \in \Delta$,
- $d_i + t \models g_i$, and
- $D_i = \{(p_j^i, (d_i + t)[0/r_j^i]) \mid j \in \{0, \dots, n_i\}\}$.

An accepting run of $\mathcal{T}$ over a timed word $\sigma = (a_0, t_0) \dots (a_n, t_n)$ is a sequence of configuration $C_0, \dots, C_{n+1}$ satisfying:

1. $C_0 = \{(s, 0)\}$
2. For each $i \in \{0, \dots, n\}$, the configuration C_{i+1} is an $(a_i, t_i - t_{i-1})$ -successor of C_i . (We use the convention that $t_{-1} = 0$.)
3. C_{n+1} contains only final states – i.e. $C_{n+1} \subseteq F \times \mathbb{R}_{\geq 0}$.

The timed language accepted by $\mathcal{T}$, denoted $L(\mathcal{T})$, is the set of timed words over Σ on which $\mathcal{T}$ has an accepting run.

As an example, the timed language over $\{a, b\}$ in which every a is followed by a b exactly one time unit later is accepted by the ATA $\mathcal{T}$ shown in the diagram below. The transition relation Δ of $\mathcal{T}$ is depicted by using an arc covering the edges corresponding to an “And” transition. For example the state s has two entries in Δ corresponding to its two outgoing transitions: $(s, b, \top, \{(\emptyset, s)\})$ and $(s, a, \top, \{(\emptyset, s), (\{x\}, t)\})$.

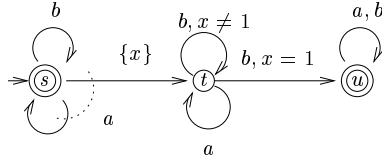


Fig. 3. An alternating timed automaton $\mathcal{T}$

We recall here the recent result of Lasota-Walukiewicz and Ouaknine-Worell:

Theorem 2 ([LW05,OW05]). *The emptiness problem of one-clock alternating timed automata over finite words is decidable.* $\square$

We now show how to translate a given ETA to a language-equivalent 1-clock ATA. The idea is fairly simple: for every guarded transition in the ETA we spawn off a copy of the automaton which verifies a literal guard of the form $\Diamond_a \in I$ or $\neg \Diamond_a \in I$. The translation is depicted in Fig. 4.

More precisely, let $\mathcal{A} = (Q, s, \delta, F)$ be an ETA over the alphabet Σ . We construct an ATA $\mathcal{T} = (Q', s', \Delta, F')$ as follows. The set of states Q' of $\mathcal{T}$ consists of the original states Q of $\mathcal{A}$, a new state u , and a new state p_c for each literal guard c (i.e. of the form $\Diamond_a \in I$ or $\neg \Diamond_a \in I$) in $\mathcal{A}$. The initial state s' is the initial state s of $\mathcal{A}$. The transition relation Δ contains the following transitions:

1. For each transition (p, a, g, q) of $\mathcal{A}$ we have the following transition in Δ . Without loss of generality let $g = c_0 \wedge \dots \wedge c_k$ be a conjunction of literal guards c_i . Then

$$(p, a, g, \{(\emptyset, q)\} \cup \{(\{x\}, p_{c_i}) \mid 0 \leq i \leq k\}) \in \Delta$$

2. For each positive literal c of the form $\Diamond_a \in I$ we have the transitions

$$\begin{aligned} (p_c, b, \top, \{(\emptyset, p_c)\}) &\in \Delta \quad (\text{for each } b \in \Sigma), \\ (p_c, a, x \in I, \{(\emptyset, u)\}) &\in \Delta. \end{aligned}$$

3. For each negative literal c of the form $\neg \Diamond_a \in I$ we have the transitions

$$\begin{aligned} (p_c, b, \top, \{(\emptyset, p_c)\}) &\in \Delta \quad (\text{for each } b \neq a), \\ (p_c, a, \neg(x \in I), \{(\emptyset, p_c)\}) &\in \Delta. \end{aligned}$$

4. $(u, a, \top, \{(\emptyset, u)\}) \in \Delta$.

The set of final states F' is given by

$$F' = F \cup \{u\} \cup \{p_c \mid c \text{ is a negative literal in } \mathcal{A}\}.$$

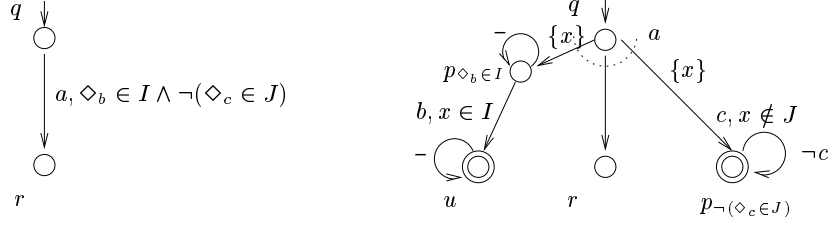


Fig. 4. Translation of an ETA transition

It is not difficult to check that $L(\mathcal{A}) = L(\mathcal{T})$. Using Theorem 2, we conclude that:

Theorem 3. *The emptiness problem for ETA's is decidable.* $\square$

4 Deciding Recursive ETA's

In this section we show how we can decide the emptiness problem for recursive ETA's by showing how they can be simulated by non-recursive ETA's, though over an extended alphabet.

To explain the idea of our “flattening” construction, consider a recursive ETA $\mathcal{A}$ that uses the floating recursive ETA's $\mathcal{B}_0, \dots, \mathcal{B}_k$ in its guards. We can then consider a timed word over the extended alphabet $\Sigma \times \{0, 1\}^{k+1}$ which represents the positions where the automata $\mathcal{B}_i$ accept the underlying timed word. Thus if $\sigma = (a_0, t_0) \dots (a_n, t_n)$ is a timed word over Σ , the canonical extension of σ (wrt $\mathcal{B}_0, \dots, \mathcal{B}_k$) is the timed word $\bar{\sigma}$ which we represent as $((a_0, v_0), t_0) \dots ((a_n, v_n), t_n)$ where for each $i \in \{0, \dots, n\}$ and $j \in \{0, \dots, k\}$, $v_i[j] = 1$ iff $\mathcal{B}_j$ accepts (σ, i) . The figure below shows the canonical extension of the timed word $(a, 0.2)(a, 1.2)(b, 1.4)(c, 2.4)$ with respect to the recursive ETA of Fig. 2.

T	0.2	1.2	1.4	2.4
Σ	a	a	b	c
$\mathcal{B}$	0	0	1	0
$\mathcal{C}$	0	0	0	1

Now the idea is essentially to run a modified version of $\mathcal{A}$, which we call $\hat{\mathcal{A}}$, over the extended timed words, and replace each atomic guard of the form $\Diamond_{\mathcal{B}_j} \in I$ in $\mathcal{A}$ by an appropriate non-recursive guard of the form $\Diamond_{(a,v)} \in I$ where $a \in \Sigma$ and v is such that $v[j] = 1$. We must further make sure that $\hat{\mathcal{A}}$ does

not accept “bad” extended words in which some 0 and 1 guesses are incorrect. For this we use ETA’s $\hat{\mathcal{B}}_j^0$ and $\hat{\mathcal{B}}_j^1$ which verify that the 0 entries (respectively the 1 entries) in the j -th row are correct.

The ETA $\hat{\mathcal{B}}_j^0$ which verifies that the 0 entries in the j -th row are correct, works by maintaining two components in its state: the first a state of $\mathcal{B}_j$ and the second a subset of states of $\mathcal{B}_j$. Whenever it gets an (a, v) with $v[j] = 0$ it pretends it has received a 1 entry instead, and pushes the resulting state to its second component. Apart from this, it runs its second component similar to a subset construction, assuming all entries are 0. Thus, if the extended word contained a 0 entry that should have been a 1, at the end of the word $\hat{\mathcal{B}}_j^0$ will have a *final* state of $\mathcal{B}_j$ and it will reject the extended word.

We now give the formal construction of the flattened ETA. Let $\mathcal{A} = (Q, s, \delta, F)$ be a recursive ETA over the alphabet Σ , and let it make use of the recursive floating ETA’s $\mathcal{B}_0, \dots, \mathcal{B}_k$, listed in the order of increasing levels. Without loss of generality (cf. Theorem 1) we assume that each $\mathcal{B}_j$ is deterministic. Let each $\mathcal{B}_j$ have the structure $(Q_j, s_j, \delta_j, F_j)$.

The required flattened ETA $\mathcal{A}'$ over the extended alphabet $\Sigma' = \Sigma \times \{0, 1\}^{k+1}$ is defined to be the intersection of the following ETA’s as below:

$$\mathcal{A}' = \hat{\mathcal{A}} \cap \bigcap_{j=0}^k \hat{\mathcal{B}}_j^0 \cap \bigcap_{j=0}^k \hat{\mathcal{B}}_j^1.$$

The ETA $\hat{\mathcal{A}}$ has the same structure as $\mathcal{A}$ and is defined to be $(Q, s, \hat{\delta}, F)$ where the transition relation $\hat{\delta}$ is given as follows. For a guard g in $\mathcal{A}$, we denote by $\hat{g}$ the guard obtained by replacing each occurrence of an atomic guard of the form $\diamond_{\mathcal{B}_j} \in I$ by the guard

$$\bigvee_{a \in \Sigma, v[j]=1} (\diamond_{(a,v)} \in I).$$

Now let $(p, a, g, q) \in \delta$. Then we have the transition $(p, (a, v), \hat{g}, q)$ in $\hat{\delta}$ for all v .

The automaton $\hat{\mathcal{B}}_j^0$ is defined as follows. $\hat{\mathcal{B}}_j^0 = (\hat{Q}_j^0, \hat{s}_j^0, \hat{\delta}_j^0, \hat{F}_j^0)$, where:

- $\hat{Q}_j^0 = Q_j \times 2^{Q_j}$
- $\hat{s}_j^0 = (s_j, \emptyset)$
- $\hat{\delta}_j^0$ is given as follows. Let (q, S) be a state of $\hat{\mathcal{B}}_j^0$, with $S = \{q_0, \dots, q_m\}$. Let $(a, v) \in \Sigma'$ with $v[j] = 0$. Then we have the transition $((q, S), (a, v), h, (r, T))$ in $\hat{\delta}_j^0$ provided the following conditions are satisfied:
 - There exists g such that $(q, (a, 0), g, r) \in \delta_j$.
 - There exists g' such that $(q, (a, 1), g', t) \in \delta_j$.
 - There exist $g_0, \dots, g_m$ such that $(q_i, (a, 0), g_i, r_i) \in \delta_j$
 - $h = \hat{g}_0 \wedge \dots \wedge \hat{g}_m \wedge \hat{g} \wedge \hat{g}'$, and,
 - $T = \{r_0, \dots, r_m\} \cup \{t\}$.

If $(a, v) \in \Sigma'$ with $v[j] = 1$, then we have the transition $((q, S), (a, v), h, (r, T))$ in $\hat{\delta}_j^0$ provided:

- There exists g such that $(q, (a, 0), g, r) \in \delta_j$.
 - There exist $g_0, \dots, g_m$ such that $(q_i, (a, 0), g_i, r_i) \in \delta_j$
 - $h = \hat{g}_0 \wedge \dots \wedge \hat{g}_m \wedge \hat{g}$, and,
 - $T = \{r_0, \dots, r_m\}$.
- $\hat{F}_j^0 = Q_j \times 2^{Q_j - F_j}$.

The ETA $\hat{\mathcal{B}}_j^1$ is defined in a similar manner, except that we now verify the 1's instead of the 0 entries.

To reason about the correctness of the construction, it is convenient to use the notion of a “projection” of an extended word to a standard timed word. Let σ' be a timed word over the extended alphabet Σ' . Let us represent σ' as (w, β, τ) where $w \in \Sigma^*$, $\beta \in (\{0, 1\}^{k+1})^*$, and $\tau \in (\mathbb{R}_{>0})^*$. Then the projection of σ' to Σ , denoted $\sigma' \downarrow \Sigma$, is defined to be the timed word (w, τ) . We extend the projection operator to work on timed languages over Σ' in the expected way.

Theorem 4. *The recursive ETA $\mathcal{A}$ accepts the projection of the language accepted by the ETA $\mathcal{A}'$, i.e $L(\mathcal{A}) = L(\mathcal{A}') \downarrow \Sigma$.*

Proof (Sketch). The proof involves two arguments. The first that the ETA over the extended alphabet $\hat{\mathcal{A}}$ accepts precisely the canonical extensions of timed words accepted by $\mathcal{A}$, *provided* we restrict ourselves to only canonical extended timed words. That is, if σ is a timed word over Σ and $\bar{\sigma}$ is its canonical extension wrt $\mathcal{B}_0, \dots, \mathcal{B}_k$, then $\sigma \in L(\mathcal{A})$ iff $\bar{\sigma} \in L(\hat{\mathcal{A}})$. This is easy to see once we observe that

$$\sigma, i \models g \text{ iff } \bar{\sigma}, i \models \hat{g}.$$

The second part of the argument is to show that the intersection of the ETA's $\hat{\mathcal{B}}_j^0$ and $\hat{\mathcal{B}}_j^1$ together ensure that $\mathcal{A}'$ accepts only canonical extensions wrt $\mathcal{B}_0, \dots, \mathcal{B}_k$. To prove this it is sufficient to argue that if σ' is an extended timed word such that for a particular j all rows below it have “correct” 0's and 1's, then $\hat{\mathcal{B}}_j^0$ (and symmetrically $\hat{\mathcal{B}}_j^1$) accepts it iff all 0 entries (respectively 1 entries) in the j -th row are correct. This is fairly routine to prove using the fact that the $\mathcal{B}_j$'s are time deterministic. $\square$

5 Expressiveness of ETA's

In this section we compare the expressiveness of the class of recursive ETA's with a couple of well known classes of timed automata in the literature. We show that recursive ETA's are incomparable in expressiveness with Alur-Dill timed automata, and that they are strictly more expressive than the class of recursive Event Predicting Automata (EPA's) of [HRS98].

The timed language L_0 of section 2 was shown to be definable by an ETA whereas it is not definable by an Alur-Dill timed automaton. Conversely, consider the language L_{ni} (for “no insertions”) over the alphabet $\{a, b\}$, in which for any two consecutive a 's the corresponding time period translated by one time unit contains no events. The complement of this language is definable by a non-deterministic timed automaton. However, as shown in [DP05], the language L_{ni}

is not definable by a 1-clock alternating timed automaton (and hence neither is its complement, since this class is closed under complement). Recursive ETA's can be seen to be a subclass of 1-clock ATA's, using Theorem 4 and the fact that 1-clock ATA's are closed under projection. It then follows that recursive ETA's cannot define the complement of L_{ni} either. Thus the class of recursive ETA's is incomparable to the class of Alur-Dill timed automata.

The class of recursive Event Predicting Automata [AFH96,HRS98] can be seen to be strictly contained in the class of recursive ETA's. Recall that EPA's use an input determined operator $\triangleright_a$ which measures the distance to the *next* a event. The EPA guard $\triangleright_a \in I$ can be simulated by the ETA guard $(\diamond_a \in I) \wedge \neg(\diamond_a \in I')$, where the interval I' is of the form $(0, l)$ if $I = [l, r]$, and $(0, l]$ if $I = (l, r)$ (here ' $\rangle$ ' is either ' $\rangle$ ' or ' $\rangle$ '). A recursive EPA can also be simulated by a recursive ETA by inductively replacing guards in a similar manner. A recursive EPA can be flattened to a projection equivalent non recursive EPA which runs over an extended alphabet. The class of non recursive EPA is shown to be a subclass of Alur-Dill timed automata [AFH94]. Since timed automata are closed under projection there exists a timed automaton which accepts the language accepted by the recursive EPA. Therefore the class of EPA's is strictly contained in the of ETA's. The figure below summarizes the arguments of this section.

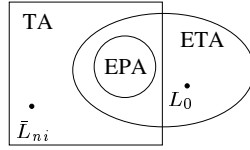


Fig. 5. Expressiveness of recursive ETA's, EPA's and timed automata

6 Decision procedures for the dual of $\diamond$

In this section we consider the class of IDA's obtained by considering the dual operator $\diamond$. The semantics of the operator is given as follows:

$$\llbracket \diamond_a \rrbracket(\sigma, i) = \{\tau(i) - \tau(j) \mid \sigma = (w, \tau), j \leq i, \text{ and } w(j) = a\}.$$

We denote this class $\text{IDA}(\diamond)$. We prove that the emptiness problem of the class of $\text{IDA}(\diamond)$ over finite words is decidable by reducing it to the emptiness problem of ETA's.

We now define the *reverse* of a timed word. Let $\sigma = (a_0, t_0) \cdots (a_n, t_n)$ be a timed word. We define the set of reversed timed words σ^R corresponding to σ as follows: $\sigma^R = \{(a_n, d - t_n) \cdots (a_0, d - t_0) \mid d > t_n\}$. For a timed language $L \subseteq T\Sigma^*$ we define $L^R = \bigcup_{\sigma \in L} \sigma^R$.

Let g be a $\diamond$ -guard, and let us denote by g^R the $\diamond$ -guard obtained from g by replacing each atomic guard $\diamond_a \in I$ in g by the atomic guard $\diamond_a \in I$. Then the following can be easily verified: Let σ and σ' be such that $\sigma' \in \sigma^R$. Then (a) if $\sigma, i \models g$ then $\sigma', (n-i) \models g^R$, and (b) if $\sigma, i \models g^R$ then $\sigma', (n-i) \models g$.

We now construct an ETA $\mathcal{A}^R$ from the given IDA($\diamond$) $\mathcal{A}$ by replacing every guard g in $\mathcal{A}$ with the guard g^R , reversing the direction of all the edges, and interchanging the start and the final states. For convenience we allow $\mathcal{A}^R$ to have multiple start states. It is then easy to verify that if $\sigma' \in \sigma^R$ then $\sigma \in L(\mathcal{A})$ implies $\sigma' \in L(\mathcal{A}^R)$, and $\sigma \in L(\mathcal{A}^R)$ implies $\sigma' \in L(\mathcal{A})$. Thus $\mathcal{A}^R$ preserves the language emptiness of $\mathcal{A}$. In general, the language accepted by $\mathcal{A}^R$ can be seen to be $L(\mathcal{A})^R$.

Since the emptiness problem for ETA's is decidable we conclude that:

Theorem 5. *The emptiness problem for IDA($\diamond$) is decidable.* $\square$

It is easy to see that recursive IDA($\diamond$) can be flattened in a similar manner to ETA's. Hence we can conclude that:

Theorem 6. *The emptiness problem for recursive IDA($\diamond$) is decidable.* $\square$

Finally, we point out that the class of IDA's with both operators $\diamond$ and $\diamond$ becomes undecidable. One can give a reduction from the halting problem of a 2-counter machine to the emptiness problem for this class, in a similar manner as done in [AD94]. More details can be found in [DM05].

Theorem 7. *The emptiness problem for IDA($\diamond, \diamond$) is undecidable.* $\square$

References

- [AD94] R. Alur and D. Dill: A theory of timed automata, *Theoretical Computer Science* **126**, Elsevier, 183–235 (1994).
- [AFH96] R. Alur, T. Feder and T.A. Henzinger: The benefits of relaxing punctuality, *J. ACM* **43**, 116–146 (1996).
- [AFH94] R. Alur, L. Fix and T. A. Henzinger: Event-clock automata: a determinizable class of timed automata, *Proc. 6th CAV*, LNCS **818**, 1–13, Springer-Verlag (1994).
- [DM05] D. D'Souza and Raj Mohan M: Eventual Timed Automata, IISc-CSA-TR-2005-8, Technical Report, Indian Institute of Science, Bangalore (2005).
- [DP05] D. D'Souza and P. Prabhakar: On the expressiveness of MTL in the pointwise and continuous semantics, IISc-CSA-TR-2005-7, Technical Report, Indian Institute of Science, Bangalore (2005).
- [DT04] D. D'Souza and N. Tabareau: On timed automata with input-determined guards, *Proc. FORMATS-FTRTFT*, LNCS **3253** (2004).
- [HRS98] T. Henzinger, J. Raskin and P. Schobbens: The regular real-time languages, *Proc. 25th ICALP*, LNCS **1443**, 580–591 (1998).
- [Koy90] R. Koymans: Specifying real-time properties with metric temporal logic, *Real Time Systems*, Vol. **2**, No.4, 255–299 (1990).
- [LW05] S. Lasota and I. Walukiewicz: Alternating timed automata, *Proc. FOSSACS 2005* (2005).
- [OW05] J. Ouaknine and J. Worrell: On the decidability of metric temporal logic, *Proc. LICS 2005* (2005).